Pontificia Universidad Javeriana · Bogotá
Fundamentos de Ingeniería de Software (12946)

Taller de Diseño en Software

Parquivo — Sistema de gestión para parqueaderos

Integrantes

Cristian Quevedo · Sergio Alexander Lara Bonilla · Jacob Riveros · Santiago Arias

Docente

Ing. Kerwin de Jesús Barros Somerson

Repositorio: github.com/Alexander-hub-co/Fundamentos_Ing_SW

Octubre de 2026

1. Introducción

Parquivo es una aplicación web para administrar parqueaderos. Cada establecimiento declara sus tarifas, convenios y horarios, y el operario de la caseta registra la entrada y la salida de los vehículos escribiendo únicamente la placa: el sistema deduce si el vehículo es carro o moto, decide si se trata de una entrada o de una salida y, al salir, calcula el cobro mostrando cómo llegó a ese valor. Varios parqueaderos comparten la plataforma con sus datos aislados entre sí.

Está construida con Next.js 16 y React 19 sobre Node.js, con PostgreSQL 16 como base de datos, Drizzle como capa de acceso a datos y Better Auth para las sesiones. El aislamiento entre establecimientos se garantiza en la propia base mediante seguridad por filas (RLS).

Este informe presenta el modelo de diseño del sistema mediante cuatro diagramas UML —análisis (ECB), clases, componentes y despliegue— construidos a partir de las tres historias de usuario más críticas del backlog:

Issue	Historia de usuario	Prioridad	Puntos	Por qué es crítica
#31	**HU-008 · Determinar entrada o salida según placa** (principal)	Alta	8	Es la operación que el operario repite todo el día; sin ella no entra ni sale ningún vehículo.
#27	HU-004 · Snapshot de tarifa y convenio al momento del cobro	Alta	8	Garantiza que un cobro hecho no cambie aunque después cambien las tarifas: es la base de la confianza en el dinero.
#30	HU-007 · Detalle del cálculo del cobro	Alta	5	Hace que cada cobro se pueda explicar al cliente, que es la propuesta de valor del producto.

Juntas cubren el ciclo completo de un vehículo: entra, el sistema calcula cuánto debe y, al salir, el cobro queda guardado tal como se hizo.

2. Gestión del trabajo: issues y commits

Se creó un sub-issue por diagrama, vinculado a la historia principal **HU-008 (#31)** en el tablero *KAMBAN_FIS_2630_G#*. Cada diagrama se cargó en la carpeta `docs/diagramas` del repositorio mediante su propio commit, que referencia el sub-issue correspondiente.

Sub-issue	Asignado	Etiquetas	Commit asociado
#62 · Diagrama de análisis (ECB)	Cristian Quevedo	Tasks · Diseño · prioridad: alta · M4 · taquilla	`docs(diagramas): diagrama de análisis ECB`
#63 · Diagrama de clases	Sergio Alexander Lara Bonilla	Tasks · Diseño · prioridad: alta · M4 · taquilla	`docs(diagramas): diagrama de clases`
#65 · Diagrama de componentes	Jacob Riveros	Tasks · Diseño · prioridad: alta · M4 · taquilla	`docs(diagramas): diagrama de componentes`
#67 · Diagrama de despliegue	Santiago Arias	Tasks · Diseño · prioridad: alta · M4 · taquilla	`docs(diagramas): diagrama de despliegue`

3. Diagrama de Análisis (ECB)

El diagrama separa el sistema en las tres clases de análisis: los **límites** (boundary) son lo que tocan los actores; los **controles** (control) contienen la lógica y orquestan; las **entidades** (entity) son los datos que persisten. Los actores sólo hablan con límites y sólo los controles tocan las entidades.

Tipo	Elementos
Actores	Operario de taquilla, Administrador de parqueadero
«boundary»	`CampoPlaca` (el único campo de la taquilla), `PanelSalida` (tiempo, tramos y convenios), `ComprobanteImpreso` (tiquete térmico), `FormularioTarifa`
«control»	`ResolverPlacaCtrl`, `RegistrarEntradaCtrl`, `CalcularCobroCtrl`, `RegistrarSalidaCtrl`, `DeclararTarifaCtrl`
«entity»	`Movimiento`, `TipoVehiculo`, `Tarifa`, `Convenio`, `PoliticaCobro`, `SesionTurno`

Recorrido por historia

- **HU-008.** El operario escribe la placa en `CampoPlaca`. `ResolverPlacaCtrl` la clasifica según `TipoVehiculo` y busca un `Movimiento` abierto con esa placa: si no existe, deriva a `RegistrarEntradaCtrl`, que crea el movimiento y lo atribuye a la `SesionTurno`; si existe, deriva a `CalcularCobroCtrl`.
- **HU-007.** `CalcularCobroCtrl` toma la `Tarifa` vigente, aplica los `Convenio` y la regla de `PoliticaCobro`, y entrega el desglose por tramos a `PanelSalida`.
- **HU-004.** Al confirmar, `RegistrarSalidaCtrl` recalcula en el instante de salida, cierra el `Movimiento` guardando una copia del cobro y emite el `ComprobanteImpreso`. Cuando el administrador declara una tarifa nueva, `DeclararTarifaCtrl` crea otra versión y los movimientos cerrados no cambian.

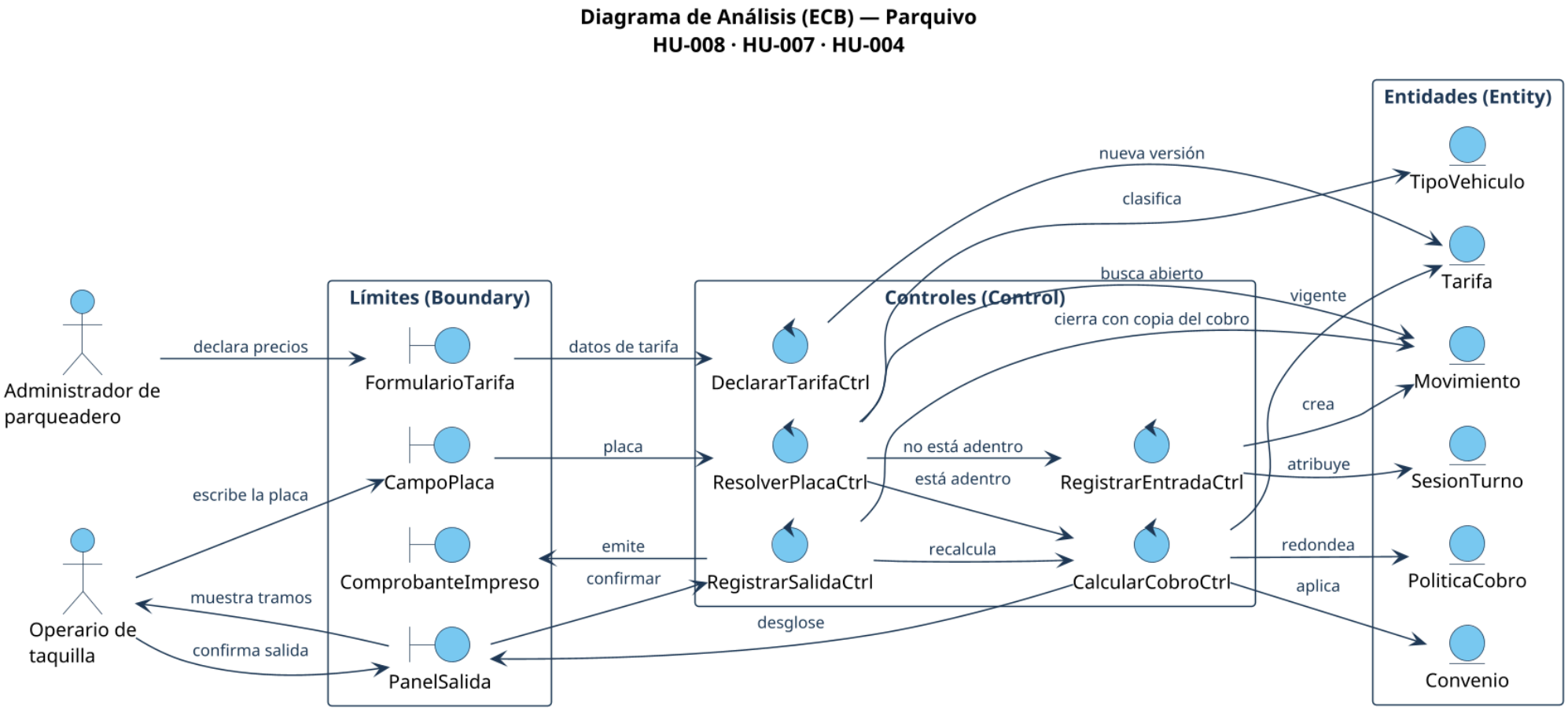


Figura 1. Diagrama de Análisis (ECB) — docs/diagramas/diagrama-ecb.md

4. Diagrama de Clases

El modelo tiene tres grupos. Los **servicios** (`ServicioTaquilla`, `CalculadoraCobro`) contienen la lógica de negocio; las **interfaces** (`ICalculadoraCobro`, `IRepositorioMovimientos`, `IRepositorioTarifas`, `IRepositorioConvenios`, `IAutorizador`) son los contratos de los que dependen; y el **dominio** (`Parqueadero`, `Movimiento`, `Tarifa`, `Convenio`, `Cobro`…) son los datos del negocio. Cada clase lleva sus atributos con tipo y visibilidad (`-` privado, `#` protegido, `+` público) y sus métodos con parámetros y tipo de retorno.

Relaciones

Tipo	Entre	Multiplicidad	Justificación
Herencia	`Tarifa` → `TarifaPorMinuto`, `TarifaPorIntervalo`, `TarifaPrimeraHoraYFraccion`	—	Los tres modelos de cobro comparten la tarifa plena y su vigencia; cada uno calcula a su manera.
Realización	`CalculadoraCobro` implementa `ICalculadoraCobro`	—	La taquilla depende del contrato, no de la implementación.
Composición	`Movimiento` ◆ `Cobro`	1 — 0..1	El cobro es la copia de HU-004: nace al cerrar el movimiento y no existe sin él.
Composición	`Cobro` ◆ `TramoCobrado` / `BeneficioAplicado`	1 — 1..* / 1 — 0..*	El desglose de HU-007 es parte del cobro.
Composición	`Movimiento` ◆ `Placa`	1 — 1	La placa normalizada pertenece al movimiento.
Agregación	`Parqueadero` ◇ `Tarifa`, `Convenio`, `Movimiento`	1 — 0..*	El establecimiento agrupa lo que declara y registra.
Asociación	`Movimiento` → `TipoVehiculo`, `Usuario`, `SesionTurno`; `Tarifa` → `TipoVehiculo`	0..* — 1 (turno 0..1)	Referencias entre objetos que viven por separado.

Principios SOLID

Principio	Cómo se aplica en el diseño
S — Responsabilidad única	Cada clase tiene un solo motivo de cambio: `Placa` normaliza y clasifica, `PoliticaCobro` sólo redondea, `CalculadoraCobro` sólo calcula y `ServicioTaquilla` sólo orquesta. En el código son módulos separados (`vehiculos/placa.ts`, `cobro/redondeo.ts`, `tarifas/calcular.ts`, `taquilla/resolver.ts`).
O — Abierto/cerrado	Un modelo de cobro nuevo se agrega como otra subclase de `Tarifa` sin modificar `CalculadoraCobro` ni `ServicioTaquilla`.
L — Sustitución de Liskov	Cualquier subclase de `Tarifa` sirve donde se espera una `Tarifa`: todas cumplen el mismo contrato de `calcular(minutos)`, que devuelve un importe entero, no negativo y que nunca supera la tarifa plena. La base lo refuerza con restricciones `CHECK`.
I — Segregación de interfaces	Hay interfaces pequeñas en lugar de un repositorio general: `ServicioTaquilla` sólo conoce `IRepositorioMovimientos` e `IAutorizador`; `CalculadoraCobro` sólo `IRepositorioTarifas` e `IRepositorioConvenios`.
D — Inversión de dependencias	`ServicioTaquilla` depende de la abstracción `ICalculadoraCobro`, no de `CalculadoraCobro`. Los servicios piden los datos a repositorios por su interfaz y no conocen PostgreSQL.

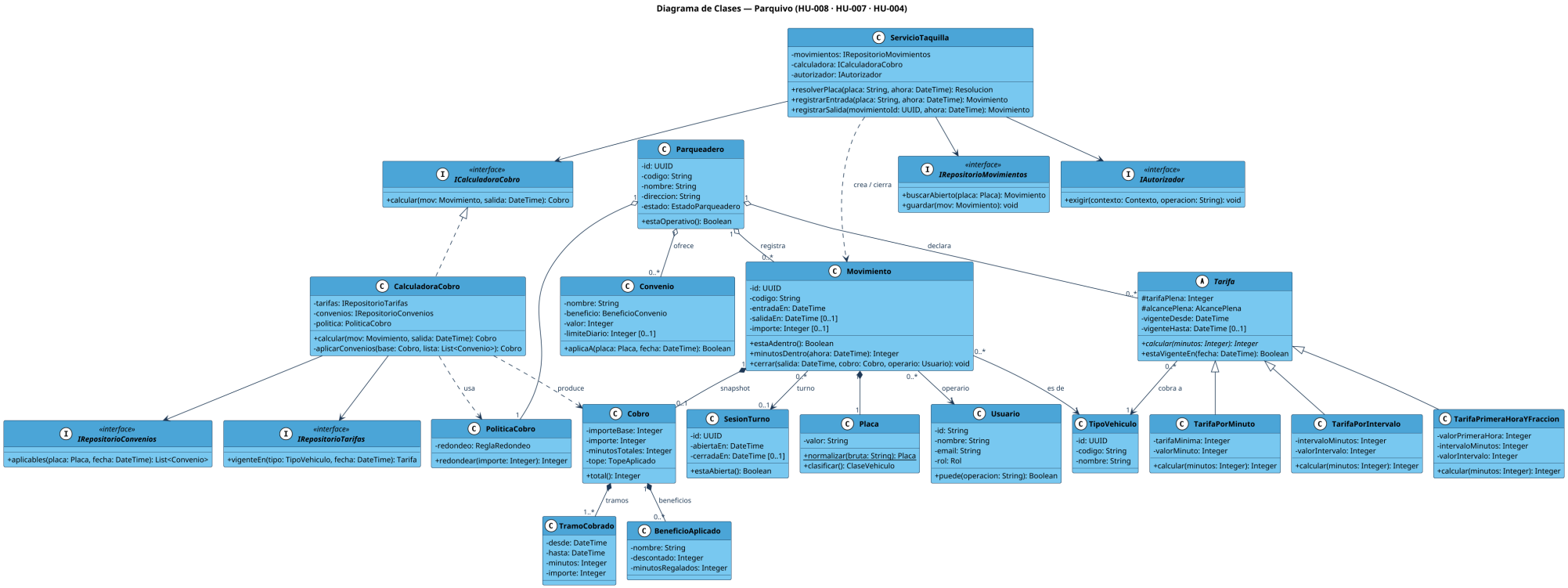


Figura 2. Diagrama de Clases — docs/diagramas/diagrama-clases.md

5. Diagrama de Componentes

Parquivo se organiza en siete componentes internos, cada uno con sus subcomponentes, y tres elementos externos que el diagrama distingue en otro color. El círculo marca la interfaz **provista** por un componente y el semicírculo la interfaz que otro **requiere**.

Componente	Subcomponentes
Interfaz de Taquilla	CampoPlaca, PanelSalida, Impresión
Interfaz de Configuración	FormularioTarifa, ProbadorCálculo
Taquilla	ResolverPlaca, RegistrarMovimiento, Comprobante
Tarifas	DeclararTarifa, TarifaVigente, CalcularImporte
Cobro y Convenios	AplicarConvenios, PolíticaRedondeo
Autenticación y Autorización	Sesión, Permisos
Persistencia	ConÁmbito (RLS), Esquema Drizzle

Interfaz	La provee	La requiere	Contrato
`ITaquilla`	Taquilla	Interfaz de Taquilla	Resolver una placa, registrar entrada y salida
`IConfigTarifas`	Tarifas	Interfaz de Configuración	Declarar una tarifa y probar el cálculo
`ICalculoCobro`	Cobro y Convenios	Taquilla	Calcular el cobro con convenios y redondeo
`ITarifaVigente`	Tarifas	Cobro y Convenios	Entregar la tarifa vigente en una fecha
`IContexto`	Autenticación	Taquilla, Tarifas	Usuario, rol y establecimiento de la sesión
`IAmbito`	Persistencia	Taquilla, Cobro, Autenticación	Transacción limitada al establecimiento

Externos: PostgreSQL 16 (base de datos, por SQL sobre TCP 5432), Better Auth (librería de sesiones y contraseñas) y la impresora térmica (dispositivo al que se imprime por el spooler del sistema operativo). Ningún componente accede a la base directamente: todo pasa por `IAmbito`, que fija el establecimiento antes de cualquier consulta para que la seguridad por filas aísle los datos de cada parqueadero.

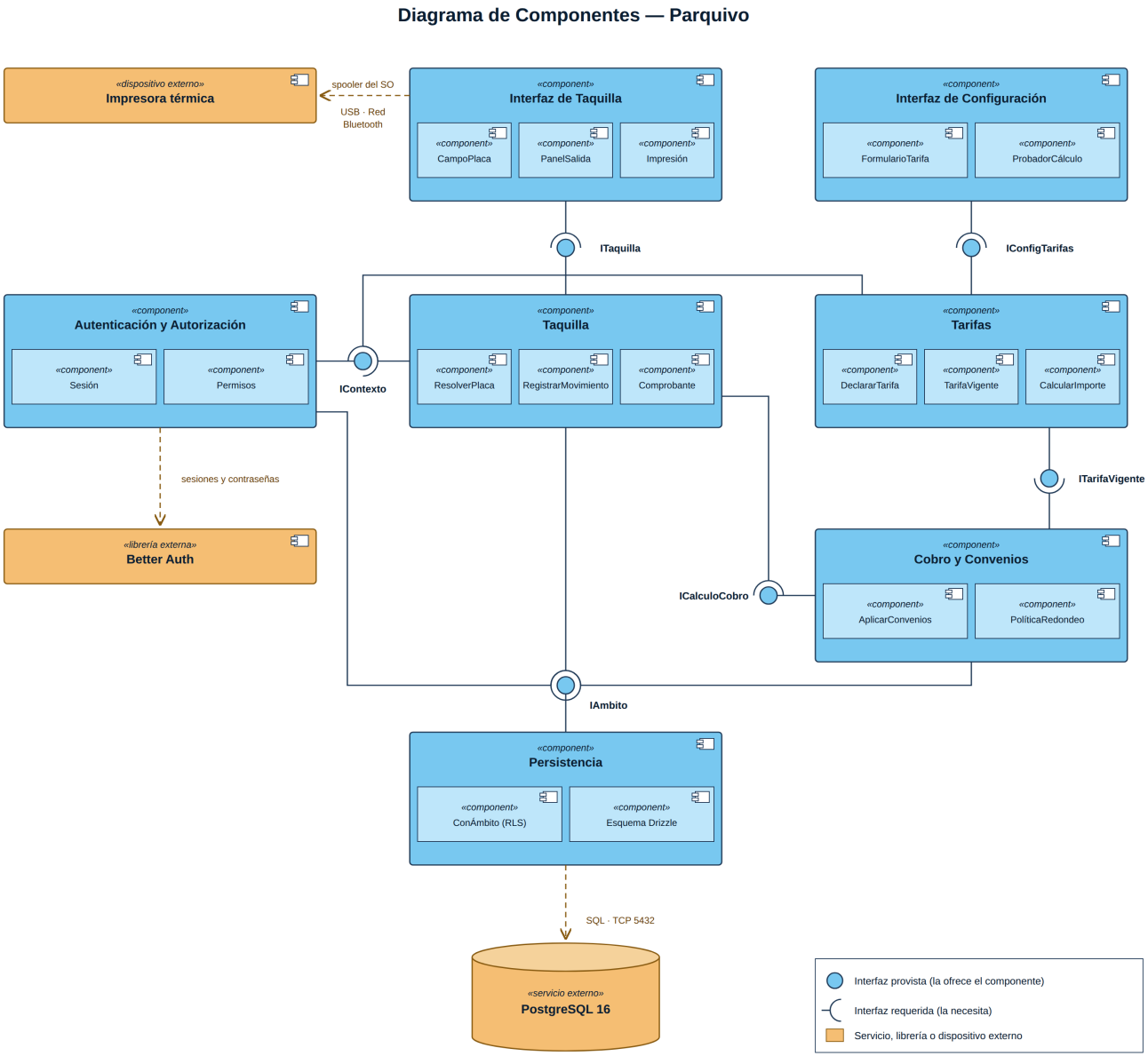


Figura 3. Diagrama de Componentes — docs/diagramas/diagrama-componentes.md

6. Diagrama de Despliegue

El sistema se despliega en cinco nodos. Los equipos del local (la PC de taquilla con su impresora térmica y la PC del administrador) usan un navegador; la aplicación corre en un servidor Node.js en una plataforma gestionada, y los datos viven en un servidor PostgreSQL.

Nodo «device»	Entorno de ejecución	Artefactos «artifact»
PC de taquilla	Chromium en modo `--kiosk-printing`	`bundle-frontend.js` (React 19 y CSS)
Impresora térmica 80 mm	—	`comprobante.html`
PC del administrador	Navegador web	`bundle-frontend.js`
Servidor de aplicación	Node.js ≥ 20.9	`parquivo-next` (compilación de Next.js 16), `migrar.js`
Servidor de base de datos	PostgreSQL 16	Esquema `parqueadero` con RLS, `roles.sql`

Comunicación	Protocolo	Puerto
Navegadores → Servidor de aplicación	HTTPS (Server Actions y React Server Components)	443
Servidor de aplicación → Base de datos	Protocolo PostgreSQL sobre TCP con TLS	5432
PC de taquilla → Impresora	Spooler del SO: USB, TCP RAW o Bluetooth	9100 (en red)

Infraestructura y hardware necesarios

- **Caseta:** un equipo capaz de ejecutar Chromium y una impresora térmica de 80 mm configurada como predeterminada.
- **Servidor de aplicación:** plataforma gestionada con Node.js 20.9 o superior y certificado TLS para el dominio.
- **Base de datos:** PostgreSQL 16 o superior con permiso para crear roles con `BYPASSRLS`, porque el aislamiento usa tres roles (`app_migrator`, `app_platform`, `app_tenant`). Si la plataforma usa agrupador de conexiones, debe estar en modo transacción.
- **Configuración:** cinco variables de entorno: las tres cadenas de conexión, `AUTH_SECRET` y `APP_URL`.

La impresión no pasa por el servidor: una impresora del local tiene dirección privada y el servidor no la alcanza, así que el tiquete sale desde el navegador de la caseta.

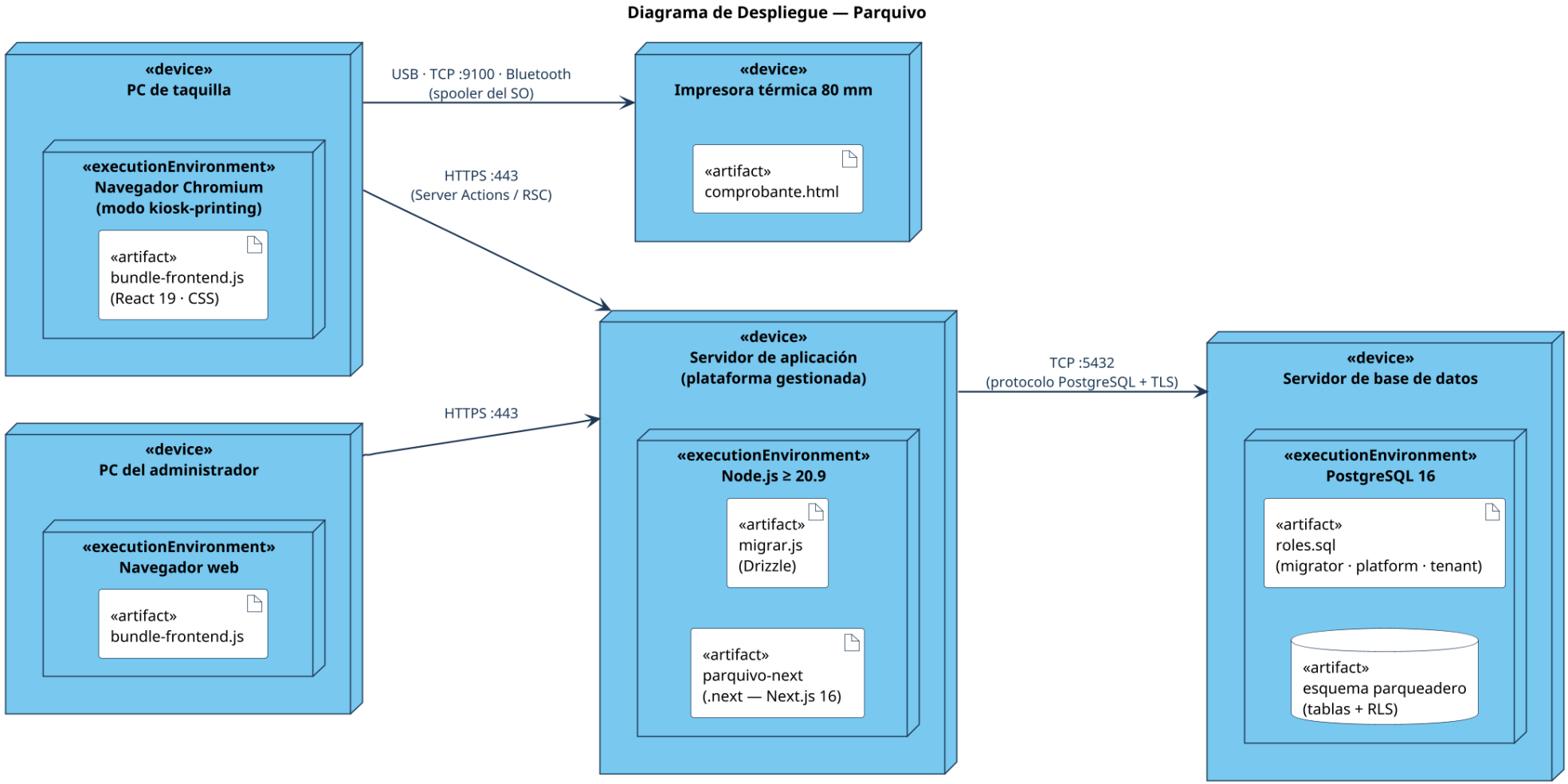


Figura 4. Diagrama de Despliegue — docs/diagramas/diagrama-despliegue.md

7. Conclusiones

- **El modelado obligó a separar decisiones que en el código estaban juntas.** Al dibujar el ECB quedó claro que «decidir si es entrada o salida» y «calcular cuánto se cobra» son controles distintos; eso permite cambiar las reglas de cobro sin tocar el flujo de la taquilla.
- **El diagrama de clases hizo visible el requisito más delicado.** Modelar el cobro como una composición del movimiento expresa HU-004 sin ambigüedad: el cobro es una copia que vive con el movimiento, no una referencia a una tarifa que puede cambiar. En un sistema que maneja dinero, esa diferencia evita reportes que no cuadran.
- **Las interfaces explican la arquitectura mejor que la estructura de carpetas.** El diagrama de componentes muestra que todo acceso a datos pasa por `IAmbito`: es el punto único donde se garantiza que un parqueadero no ve los datos de otro.
- **El despliegue reveló restricciones que no son de código.** La impresora sólo es alcanzable desde la caseta y la base necesita permisos para crear roles; ambas condiciones limitan qué proveedores sirven y deben verificarse antes de elegir uno.
- **Los principios SOLID dejaron de ser teoría.** La jerarquía de tarifas (abierto/cerrado y Liskov) y la dependencia de la taquilla en `ICalculadoraCobro` (inversión de dependencias) son las piezas que permiten agregar un modelo de cobro nuevo sin reescribir lo que ya funciona.

En conjunto, los cuatro diagramas dan una vista del sistema que un integrante nuevo puede recorrer sin leer el código: qué hace cada historia (ECB), cómo se estructura el dominio (clases), cómo se dividen las responsabilidades (componentes) y dónde corre cada pieza (despliegue).